

**INSTITUTO FEDERAL DE EDUCAÇÃO CIÊNCIA E TECNOLOGIA DE SÃO
PAULO**

**CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS INTEGRADO AO
ENSINO MÉDIO**



SABOR E CLIC

Documentação do Projeto Integrador do Terceiro Bimestre

São Paulo

2026

SUMÁRIO

- 1 Apresentação do projeto
- 2 Objetivos e escopo
- 3 Concepção do sistema e integração entre disciplinas
- 4 Atores e responsabilidades
- 5 Fluxo geral do produto
- 6 Reserva
- 7 Sessão
- 8 Cardápio e carrinho
- 9 Pedidos
- 10 KDS
- 11 Administração de Bancadas e catálogo
- 12 Arquitetura da aplicação
- 13 Lógica JavaScript
- 14 Arquitetura Flask e rotas essenciais
- 15 Banco de dados e DER
- 16 Segurança e níveis de acesso
- 17 Identidade visual e prototipação
- 18 Requisitos funcionais e não funcionais
- 19 Tecnologias utilizadas
- 20 O que concluímos e próximas etapas
- 21 Atendimento aos requisitos do terceiro bimestre
- 22 Estado atual da implementação e integração
- Referências

1 APRESENTAÇÃO DO PROJETO

Nós desenvolvemos o Sabor e Clic como uma proposta de sistema para organizar a operação de uma cozinha compartilhada com Bancadas reserváveis e atendimento gastronômico integrado. Nosso ponto de partida foi um problema recorrente: quando Reservas, disponibilidade de espaços, Cardápio e Pedidos são controlados por planilhas ou anotações isoladas, surgem conflitos de horário, falta de rastreabilidade e dificuldade para acompanhar a produção. Por isso, concebemos uma experiência única que liga o Cliente à cozinha e à administração.

Nossa documentação acompanha o fluxo do produto: Reserva → Sessão → Cardápio e Pedidos → KDS. Escolhemos esse encadeamento porque cada Pedido precisa existir dentro de um contexto de atendimento. A Reserva define quem utilizará qual Bancada e em qual período; a Sessão representa o uso efetivo dessa Reserva; o Cardápio permite selecionar itens; o Pedido registra a solicitação; e o KDS oferece à cozinha uma visão organizada da produção.

Ao longo do terceiro bimestre, reunimos contribuições de Artes Visuais e Design, JavaScript, Python, Banco de Dados e Segurança. Nosso protótipo estático demonstra as principais telas e interações. A modelagem de dados descreve as entidades e os relacionamentos necessários para sustentar a aplicação. Também iniciamos a estrutura de back-end e definimos a arquitetura teórica das rotas que completarão a integração.

2 OBJETIVOS E ESCOPO

2.1 Objetivo geral

Nosso objetivo é construir um sistema web que permita administrar Bancadas gastronômicas, criar Reservas, iniciar Sessões, consultar o Cardápio, enviar Pedidos e acompanhar seu preparo no KDS. Pretendemos centralizar dados que normalmente estariam dispersos e oferecer uma visão adequada para Cliente, Cozinheiro e Administrador.

2.2 Objetivos específicos

- Representar o fluxo completo do atendimento de forma clara e navegável.
- Modelar as entidades, atributos e relacionamentos em um DER e em um dicionário de dados.
- Aplicar JavaScript moderno para manipular o DOM, manter o carrinho e calcular valores em memória.
- Definir uma API Flask organizada por responsabilidades, com comunicação por JSON.
- Planejar autenticação, validação e RBAC para proteger ações e dados.
- Criar identidade visual coerente e interfaces para Cardápio Digital e KDS.

2.3 Limites desta etapa

Nesta etapa concluímos o protótipo estático e a documentação conceitual de dados. Iniciamos o back-end, mas a aplicação Flask integrada ao banco ainda faz parte da evolução do projeto. Por esse motivo, quando descrevemos uma rota completa ou uma regra transacional, apresentamos a arquitetura que definimos para a implementação, distinguindo-a das interações que já funcionam localmente no protótipo.

3 CONCEPÇÃO DO SISTEMA E INTEGRAÇÃO ENTRE DISCIPLINAS

Nós organizamos o projeto como um sistema interdisciplinar. Design define como o usuário compreende estados e ações; JavaScript transforma a interface em uma experiência dinâmica; Flask centraliza regras e comunicação; Banco de Dados garante persistência e relacionamentos; Segurança controla identidade, permissão e validação. Nenhuma dessas partes resolve isoladamente o problema: a qualidade do fluxo depende da concordância entre todas elas.

Disciplina	Contribuição realizada	Evolução arquitetural
Design	Paleta, tipografia, componentes, telas por perfil, Cardápio e KDS.	Fidelidade ao protótipo, acessibilidade e testes de usabilidade.
JavaScript	JavaScript modularizado por responsabilidade, DOM, carrinho, validações, cálculos em memória e perfis simulados.	Classes ES6+, Fetch/JSON, async/await e integração real com Flask no quarto bimestre.
Python e Flask	Estrutura Flask inicial com controllers e endpoints simulados para autenticação, Reservas, Cardápio, Pedidos e KDS.	MVC completo, services, persistência, autenticação/RBAC e integração com o banco.
Banco de Dados	DER, MER, EER e dicionário de dados.	Modelo físico, scripts SQL, consultas e triggers.
Segurança	Definição de perfis e separação visual.	Autenticação, sessão, RBAC no servidor e sanitização.

4 ATORES E RESPONSABILIDADES

Ator	Objetivo no sistema	Principais ações
Cliente	Utilizar uma Bancada e consumir durante uma Sessão.	Consultar Bancadas, reservar, abrir Sessão, montar carrinho, enviar e acompanhar Pedidos.
Cozinheiro	Organizar a produção.	Consultar fila, iniciar preparo, marcar Pedido pronto e entregue, acompanhar tempos.
Administrador	Configurar e supervisionar a operação.	Gerenciar Bancadas, Reservas, Pratos, Ingredientes, usuários e relatórios.

Nós representamos os três atores por menus distintos. Essa separação melhora a compreensão do protótipo, mas sabemos que visibilidade de menu não equivale a autorização. Na aplicação integrada, cada requisição protegida precisará ser validada pelo Flask de acordo com o usuário autenticado e seu papel.

5 FLUXO GERAL DO PRODUTO

O fluxo começa quando o Cliente consulta uma data e um intervalo. O sistema compara capacidade e disponibilidade das Bancadas. Com uma opção válida, cria a Reserva e preserva o período e o valor acordado. No início do atendimento, a Reserva gera uma Sessão única. É a Sessão ativa que autoriza o acesso ao Cardápio e a criação de Pedidos. A cozinha recebe o Pedido no KDS, altera seu estado e devolve esse progresso ao Cliente. No encerramento, a Sessão deixa de aceitar novos Pedidos e conserva o histórico.

Etapa	Estado de entrada	Processamento	Saída
Reserva	Bancada disponível.	Validamos período, capacidade e conflito.	Reserva confirmada.
Sessão	Reserva válida.	Criamos contexto único e ativamos o atendimento.	Sessão ativa.
Cardápio	Sessão ativa.	Exibimos Pratos disponíveis e mantemos carrinho.	Itens selecionados.
Pedido	Carrinho válido.	Revalidamos itens, preços e quantidades.	Pedido recebido.
KDS	Pedido recebido.	Organizamos fila e controlamos transições.	Pedido em preparo, pronto e entregue.

Esse fluxo também orienta a estrutura de dados. O Pedido não precisa repetir Cliente ou Bancada, porque sua Sessão conduz à Reserva e a Reserva identifica ambos. Essa decisão reduz redundância e mantém uma única fonte para o contexto do atendimento.

6 RESERVA

6.1 Lógica de negócio

Nós tratamos a Reserva como o compromisso de uso de uma Bancada durante um intervalo. Para criar uma Reserva, precisamos identificar o Cliente, a Bancada, o início, o fim, o número de pessoas e o valor. A Bancada deve estar ativa, comportar o grupo e não possuir outra Reserva incompatível. O fim precisa ser posterior ao início.

A condição abstrata de conflito é: há sobreposição quando o início solicitado ocorre antes do fim existente e o fim solicitado ocorre depois do início existente. Essa regra deve ser executada no servidor imediatamente antes da gravação. A tela pode antecipar a disponibilidade, mas somente o back-end pode confirmar, pois outra pessoa pode reservar o mesmo intervalo enquanto o primeiro Cliente ainda preenche o formulário.

6.2 Contribuição de cada disciplina

Em Design, criamos cartões de Bancada, etiquetas de disponibilidade, filtros e formulário de Reserva. Em JavaScript, utilizamos o array benches e templates para renderizar capacidade, recursos, preço e estado. Em Banco de Dados, relacionamos CLIENTE e BANCADA à RESERVA. Em Flask, definimos a necessidade de rotas de consulta, criação, alteração e cancelamento. Em Segurança, estabelecemos que o Cliente só pode manipular suas próprias Reservas, enquanto o Administrador possui acesso operacional mais amplo.

```
const benches = [  
  {id:1, nome:"Bancada Toscana", cap:4, preco:85,  
    status:"Disponível", recursos:["Cooktop","Pia","Tomadas"]}  
];
```

Trecho 1 - Estrutura real utilizada para representar Bancadas no protótipo.

No protótipo, os dados permanecem em memória e os botões simulam confirmação. Na aplicação final, o JavaScript enviará os campos em JSON, exibirá estado de carregamento e tratará respostas 201, 400 ou 409 sem recarregar a página.

7 SESSÃO

Nós adotamos Sessão como entidade central do atendimento. Uma Reserva descreve o período contratado; a Sessão registra o uso efetivo e reúne os Pedidos daquele período. Cada Reserva deve gerar exatamente uma Sessão, protegida por uma relação um para um. A Sessão possui código de acesso, estado e datas de criação, início e encerramento.

Uma Sessão pode estar criada, ativa ou encerrada. Somente ativa aceita Pedidos. A ativação registra o início real; o encerramento invalida o código de acesso e bloqueia novas solicitações. Essa verificação precisa acontecer no Flask a cada Pedido, pois não podemos confiar apenas no botão da interface.

Na tela que desenvolvemos, mostramos tempo restante, quantidade de Pedidos, consumo, pessoas, Pedidos vinculados e ações de navegação. Em JavaScript, essa tela é uma função do objeto views. No banco, SESSAO recebe uma referência única à RESERVA e origina vários PEDIDOS. Em Segurança, o acesso deve ser limitado ao Cliente responsável, à operação autorizada e ao Administrador.

8 CARDÁPIO E CARRINHO

Nosso Cardápio Digital apresenta foto, nome, descrição, categoria, preço e disponibilidade. Criamos oito produtos demonstrativos e organizamos a experiência em Entradas, Pratos, Bebidas e Sobremesas. Quando um item está indisponível, desabilitamos seu botão de adição. Essa regra visual ajuda o usuário, mas o servidor deverá revalidá-la na confirmação.

8.1 Lógica do carrinho

```
function addCart(id){
  const p=products.find(x=>x.id===id);
  if(p.status!=="Disponível") return;
  const ex=state.cart.find(x=>x.id===id);
  if(ex) ex.qtd++;
  else state.cart.push({id:p.id,nome:p.nome,preco:p.preco,qtd:1});
}
function cartTotal(){
  return state.cart.reduce((s,i)=>s+i.preco*i.qtd,0);
}
```

Trecho 2 - Adição de item e cálculo do total no protótipo.

Primeiro localizamos o produto pelo identificador. Se ele estiver indisponível, encerramos a função. Depois procuramos o mesmo item no carrinho: quando existe, incrementamos a quantidade; quando não existe, adicionamos um novo objeto. O total é calculado com `reduce`, acumulando preço multiplicado pela quantidade. Essa lógica satisfaz o objetivo do terceiro bimestre de manipular arrays, objetos e cálculos em memória.

Na integração, o navegador enviará identificadores e quantidades, mas não será responsável pelo valor final. O Flask consultará os preços atuais, validará cada Prato e calculará os subtotais. Isso evita que um preço alterado no console do navegador seja aceito como verdadeiro.

9 PEDIDOS

O Pedido agrupa itens solicitados durante uma Sessão. Definimos que ele deve possuir ao menos um ITEM_PEDIDO, estado, valor total, tempo estimado e data de criação. ITEM_PEDIDO preserva quantidade, preço unitário praticado e subtotal. Mesmo que o preço do catálogo mude posteriormente, o histórico continua representando a compra original.

Ao confirmar, o servidor deverá verificar autenticação, propriedade da Sessão, estado ativo, lista não vazia, quantidades positivas e Pratos disponíveis. Em seguida, buscará os preços, calculará subtotais, gravará Pedido e itens na mesma transação e devolverá JSON. Se qualquer etapa falhar, a transação inteira deve ser desfeita.

Estado	Significado	Próxima ação esperada
AGUARDANDO ou RECEBIDO	Pedido aceito e visível para a cozinha.	Iniciar preparo ou cancelar conforme regra.
EM PREPARO	Produção iniciada.	Marcar como pronto.
PRONTO	Pedido finalizado pela cozinha.	Marcar como entregue.
ENTREGUE	Ciclo operacional concluído.	Somente consulta histórica.
CANCELADO	Pedido interrompido.	Somente consulta ou procedimento administrativo.

10 KDS

O Kitchen Display System é o painel do Cozinheiro. Nós o organizamos em colunas para tornar o estado imediatamente visível: Recebidos, Em preparo e Prontos. Cada ticket apresenta número, Bancada, tempo e itens. Cores e etiquetas reforçam prioridade e atraso. Essa estrutura reduz a necessidade de abrir cada Pedido separadamente.

No protótipo, os tickets continuam sendo exemplos visuais, mas o back-end simulado já define uma consulta específica para o KDS. Nesta primeira entrega, a interface demonstra a fila e os estados sem sincronização real. No quarto bimestre, o KDS passará a consumir Pedidos persistidos pela API e enviará alterações de estado ao Flask, que validará as transições antes de atualizar o banco.

Design contribui com hierarquia, cores e leitura rápida. JavaScript controla renderização e atualização sem F5. Flask organiza fila e transições. Banco registra estado e horários. Segurança limita as ações a Cozinheiro e Administrador. Assim, o KDS reúne todas as disciplinas em uma única função operacional.

11 ADMINISTRAÇÃO DE BANCADAS E CATÁLOGO

O Administrador configura os recursos utilizados nos fluxos anteriores. Desenvolvemos telas para Bancadas, Reservas, Produtos, Ingredientes, substituições, usuários e relatórios. As operações exibidas ainda são demonstrativas, mas definem a experiência e os dados necessários para os futuros CRUDs.

A exclusão de Prato deve ser lógica por meio do atributo ativo. Dessa forma, um item deixa de aparecer para novas compras sem destruir ITEM_PEDIDO histórico. A ficha técnica relaciona Prato e Ingrediente com quantidade necessária. A substituição é direcional e possui prioridade e fator de conversão. A futura disponibilidade poderá somar o consumo de todos os itens e escolher substitutos autorizados quando o ingrediente original estiver insuficiente.

12 ARQUITETURA DA APLICAÇÃO

Nós definimos uma arquitetura em camadas. O navegador contém HTML, CSS e JavaScript; a comunicação utiliza HTTP e JSON; o Flask recebe requisições, autentica, valida e executa regras; a camada de dados persiste entidades relacionais. O banco nunca deve ser acessado diretamente pelo navegador.

Camada	Responsabilidade	Exemplo
View	Apresentar conteúdo e capturar eventos.	Cardápio, formulário e KDS.
JavaScript de página	Gerenciar estado visual e chamar API.	Carrinho, filtros e feedback.
Routes Flask	Interpretar HTTP, parâmetros e códigos de resposta.	POST /api/reservas.
Controllers e services	Executar caso de uso e regras.	Verificar conflito e criar Sessão.
Models e repositórios	Representar e consultar dados.	Reserva, Sessão, Pedido e Prato.
Banco relacional	Garantir persistência e integridade.	PK, FK, unicidade e transações.

O repositório já separa parte das responsabilidades do front-end em arquivos JavaScript específicos e possui uma estrutura Flask inicial. No navegador, dados simulados sustentam as telas e os fluxos. No servidor, controllers exemplificam contratos HTTP sem persistência definitiva. A direção arquitetural continua sendo uma aplicação em camadas, na qual o front-end comunica-se apenas com Flask e o banco permanece acessível somente pelo back-end.

13 LÓGICA JAVASCRIPT

O JavaScript atual foi dividido por responsabilidade para evitar um arquivo monolítico e tornar explícitos os conteúdos do terceiro bimestre. `data.js` concentra arrays, objetos literais e estado compartilhado; `site.js` reúne navegação, `localStorage` e `helpers`; `components.js` gera componentes visuais; `catalog.js` manipula dinamicamente o DOM do Cardápio; `cart.js` mantém o carrinho e calcula valores; `validation.js` valida Reservas; `login.js` trata o login demonstrativo; `views.js` registra as telas; e `ally.js` concentra comportamentos de acessibilidade. A navegação ocorre entre páginas HTML separadas, caracterizando uma MPA.

```
function renderCatalog(category = "Todos") {  
  const grid = document.querySelector(".grid.cols-4");  
  grid.innerHTML = "";  
  products.forEach(product => {  
    if (category !== "Todos" && product.cat !== category) return;  
    const wrapper = document.createElement("div");  
    wrapper.innerHTML = productCard(product).trim();  
    grid.appendChild(wrapper.firstElementChild);  
  });  
}
```

Trecho 3 - Renderização dinâmica do Cardápio utilizando `forEach` e manipulação do DOM.

Utilizamos arrays, objetos literais, `const`, arrow functions, template literals, `find`, `filter`, `reduce`, `forEach`, `for...of` e `for...in`. O Cardápio é atualizado dinamicamente pelo DOM; o carrinho mantém itens e calcula subtotais e total em memória; os formulários possuem validações simples; e o login demonstrativo seleciona o perfil e armazena a sessão localmente. Esses elementos cobrem os conteúdos de JavaScript previstos para o terceiro bimestre.

No quarto bimestre, a modularização será ampliada com classes ES6+, encapsulamento e módulos voltados à comunicação com a API. As chamadas `Fetch` serão assíncronas, utilizarão JSON e serão tratadas com `async/await` e `try/catch`. O estado que hoje é simulado no navegador passará a refletir as respostas reais do Flask.

14 ARQUITETURA FLASK E ROTAS ESSENCIAIS

O back-end do terceiro bimestre é propositalmente simulado. Ele já possui ponto de inicialização, configuração da aplicação e controllers para autenticação, Reservas e Pedidos. Esses arquivos demonstram endpoints fundamentais e separação de responsabilidades, enquanto os models permanecem simples e sem persistência definitiva. A finalidade desta etapa é validar os contratos das operações antes da implementação completa do MVC, do banco e da integração com o front-end.

Método e rota	Atores	Lógica essencial
POST /api/cadastro	Cliente	Cadastro simulado.
POST /api/login	Todos	Login simulado por perfil.
GET /api/bancadas	Cliente/Admin	Lista Bancadas simuladas.
POST /api/reservas	Cliente/Admin	Cria Reserva simulada.
GET /api/reservas/{id}	Cliente/Admin	Consulta Reserva por ID.
GET /api/cardapio	Cliente	Retorna Cardápio simulado.
POST /api/pedido/cadastro	Cliente	Cria Pedido simulado.
GET /api/pedido/{id}	Cliente/Admin	Consulta Pedido por ID.
GET /api/kds/{id}	Cozinheiro/Admin	Retorna Pedido para o KDS.

14.1 Exemplo atual de rota simulada

```
# POST /api/reservas - exemplo simplificado
def post_reserva(dados):
    cliente = dados["cliente"]
    bancada = dados["bancada"]
    Reserva_model.post_reserva(cliente, bancada)
    return True, 200
```

Trecho 4 - Exemplo do back-end simulado para criação de Reserva.

O exemplo demonstra o contrato básico de uma operação: receber dados, encaminhá-los para a camada responsável e devolver um resultado com código HTTP. Nesta primeira entrega, os models são simulados. Na implementação definitiva, controllers e services aplicarão autenticação, validações, regras de negócio e transações antes de acessar o banco de dados.

15 BANCO DE DADOS E DER

Nós modelamos USUARIO e suas especializações CLIENTE, COZINHEIRO e ADMINISTRADOR; BANCADA e RESERVA; SESSAO, PEDIDO e ITEM_PEDIDO; PRATO e INGREDIENTE; além de FICHA_TECNICA e SUBSTITUIDO_POR. O dicionário descreve atributos, tipos sugeridos, domínios e regras de consistência.

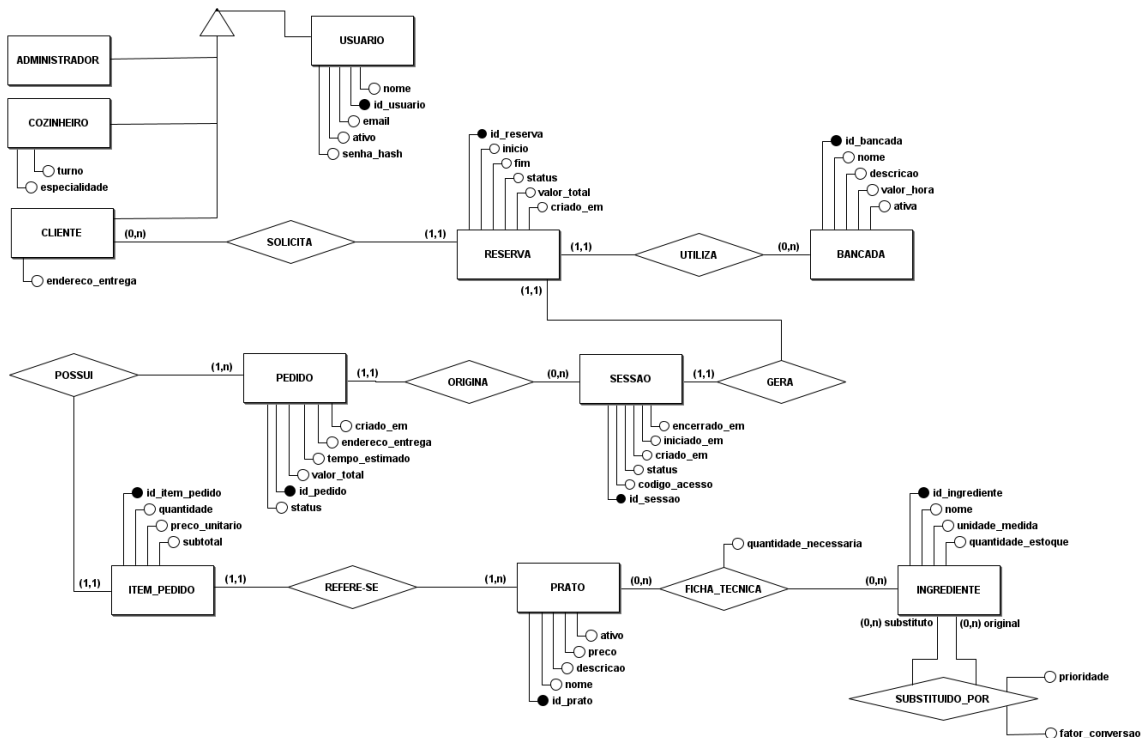


Figura 1 - Diagrama Entidade Relacionamento do Sabor e Clic. Fonte: elaboração da equipe, 2026.

A cadeia RESERVA ↔ SESSAO → PEDIDO → ITEM_PEDIDO é o núcleo do modelo. CLIENTE solicita RESERVA e BANCADA é utilizada pela RESERVA. Cada item referencia um PRATO. FICHA_TECNICA resolve a relação muitos para muitos entre Prato e Ingrediente, enquanto SUBSTITUIDO_POR utiliza duas referências para Ingrediente.

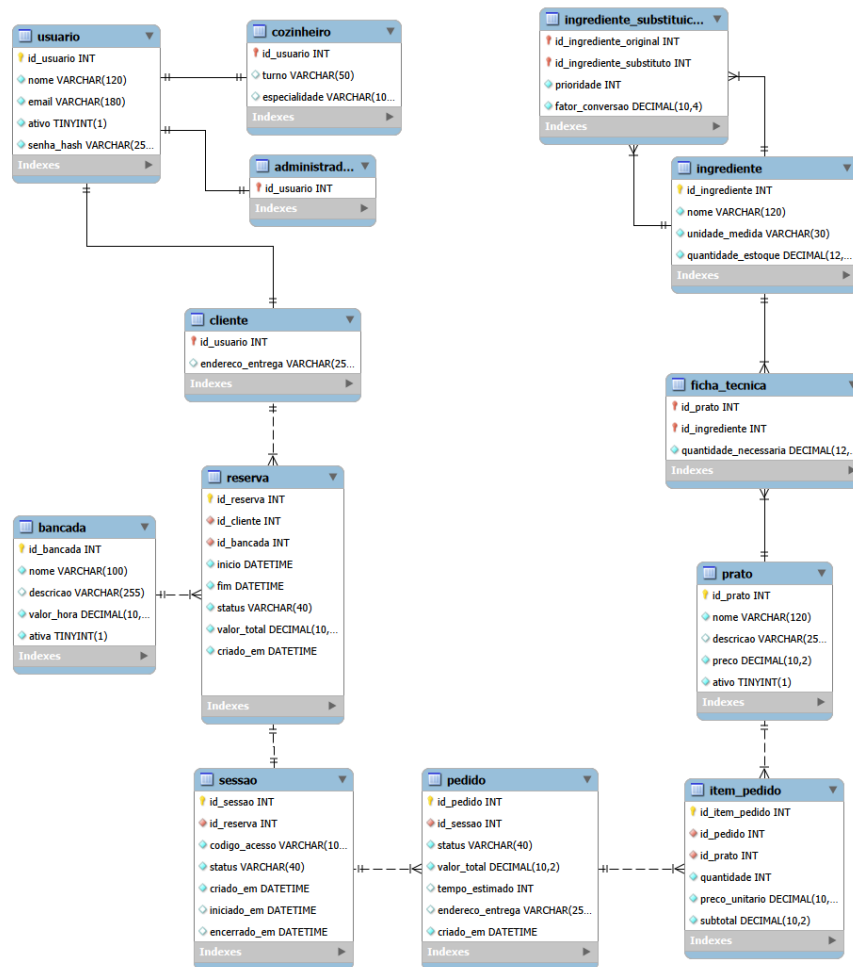


Figura 2 - Modelo de entidades e relacionamentos. Fonte: elaboração da equipe, 2026.

Definimos regras de integridade: e-mail único, valores não negativos, fim posterior ao início, Sessão única por Reserva, quantidade positiva, Pedido com itens e chaves estrangeiras obrigatórias. No quarto bimestre, o modelo físico deverá traduzir essas regras para SQL, constraints, índices, inserções básicas, consultas e triggers de estoque.

16 SEGURANÇA E NÍVEIS DE ACESSO

Definimos RBAC com três papéis. Cliente acessa seus próprios dados; Cozinheiro opera Pedidos e KDS; Administrador gerencia cadastros e visão geral. No protótipo, essa divisão aparece nos menus. Na aplicação final, ela será aplicada no servidor por decorators ou middleware, porque ocultar um botão não impede uma chamada HTTP direta.

Risco	Decisão arquitetural
Senha exposta	Armazenar hash forte com salt; nunca senha em texto ou cifra reversível.
Preço manipulado	Recalcular total no Flask a partir do catálogo.
Reserva duplicada	Validar sobreposição dentro de transação.
SQL Injection	Utilizar ORM ou consultas parametrizadas.
XSS	Escapar dados e preferir textContent para conteúdo externo.
CSRF/cookie	Configurar HttpOnly, Secure, SameSite e proteção CSRF quando aplicável.
Acesso indevido	Verificar identidade, propriedade do recurso e papel em cada rota.

O utilitário Fernet existente pode servir para dados que precisem ser recuperados, mas não para senha. Para senha, adotaremos hash unidirecional. Também planejamos logs sem informações sensíveis, mensagens genéricas para falhas de login e validação duplicada no cliente e no servidor.

17 IDENTIDADE VISUAL E PROTOTIPAÇÃO

Nós construímos uma identidade inspirada em materiais e cores gastronômicas. Utilizamos grafite #2B241E para estrutura, fundo #F6F1E8, cobre #BB6B2F como destaque, verde #4B7A52 para sucesso, açafrão #B9832A para atenção e páprica #A6432E para erro. Archivo aparece em títulos e Hanken Grotesk nos elementos de interface.

Criamos componentes consistentes: menu lateral, cartões, badges, botões, tabelas, filtros, indicadores, timeline e colunas do KDS. CSS Grid e Flexbox organizam o conteúdo, enquanto media queries adaptam a interface. As imagens das Bancadas e dos produtos aproximam o protótipo de uma experiência de uso real.

Ainda precisamos incorporar ao repositório os wireframes e o link/exportação do protótipo Figma. O front-end atual funciona como protótipo navegável codificado e demonstra telas de Cliente, Cozinheiro e Administrador.

[INSERIR FIGURA - WIREFRAMES E PROTÓTIPO FIGMA]

18 REQUISITOS FUNCIONAIS E NÃO FUNCIONAIS

Código	Requisito funcional	Estágio
RF01	Entrar e acessar interface correspondente ao papel.	Demonstrado no protótipo; autenticação real é próxima etapa.
RF02	Consultar e reservar Bancadas.	Interface concluída; persistência planejada.
RF03	Criar, ativar e encerrar Sessão.	Modelado e prototipado.
RF04	Consultar Cardápio e manter carrinho.	Implementado em memória.
RF05	Criar e acompanhar Pedido.	Interface concluída; API planejada.
RF06	Operar fila no KDS.	Layout concluído; atualização persistente planejada.
RF07	Administrar catálogo, Ingredientes e usuários.	Telas demonstrativas concluídas; CRUD planejado.

Requisito não funcional	Como tratamos
Usabilidade	Hierarquia, feedback, navegação por perfil e consistência visual.
Responsividade	Grid, Flexbox e media queries no CSS.
Manutenibilidade	Planejamento modular de pages, models, api, routes e services.
Segurança	RBAC, validação no servidor, hash e consultas parametrizadas.
Integridade	Transações, PK/FK, unicidade e domínios de estado.
Desempenho	Filtros no servidor, índices futuros e atualização parcial do DOM.

19 TECNOLOGIAS UTILIZADAS

Tecnologia	Aplicação no projeto
HTML5	Estrutura das páginas e formulários.
CSS3	Identidade visual, componentes e responsividade.
JavaScript ES6+	DOM, estado, arrays, templates, carrinho e cálculos.
JSON e Fetch API	Formato e mecanismo definidos para integração; exemplo inicial no back-end.
Python e Flask	Base escolhida para API, regras e autenticação.
Banco relacional/MySQL Workbench	DER, MER, EER e futura persistência SQL.
Git e GitHub	Versionamento e integração do trabalho da equipe.

20 O QUE CONCLUÍMOS E PRÓXIMAS ETAPAS

20.1 O que concluímos

Concluímos o protótipo navegável, a identidade visual, as telas dos três atores, o Cardápio, o carrinho em memória, a visualização de Pedidos, o layout do KDS, o DER, o MER, o arquivo EER e o dicionário de dados. Também modularizamos o JavaScript para evidenciar arrays, objetos, for...of, for...in, forEach, DOM, validações e cálculos em memória, e estruturamos um back-end Flask simulado com endpoints fundamentais.

20.2 O que iniciamos

Implementamos uma primeira estrutura Flask com configuração da aplicação e controllers simulados para cadastro/login, Bancadas/Reservas e Cardápio/Pedidos/KDS. Esses endpoints ainda não formam uma API persistente, mas já estabelecem contratos compatíveis com a evolução do sistema. A autenticação real, o banco físico e a comunicação Fetch/JSON permanecem para a próxima etapa.

20.3 Próximas etapas

Nas próximas etapas, completaremos a factory Flask e a separação MVC, substituiremos os models simulados por persistência real, implementaremos autenticação e RBAC, criaremos a Sessão persistente, integraremos Cardápio, Pedido e KDS ao banco e conectaremos o front-end à API por Fetch/JSON. Depois serão acrescentados tratamento de loading/erros e testes para conflitos, permissões, totais e transições de estado.

21 ATENDIMENTO AOS REQUISITOS DO TERCEIRO BIMESTRE

Requisito	Situação	Evidência produzida
DER	Concluído	DER_saborclic.png, MER_saborclic.png e EER_saborclic.mwb.
Dicionário de dados	Concluído	Documento em DOCX e PDF com entidades, atributos e relações.
Wireframes/Figma	Em complementação	Protótipo codificado existe; arquivo ou link Figma deverá ser incluído.
Identidade visual	Concluída	Paleta, tipografia, marca e componentes em CSS.
Cardápio Digital	Concluído como protótipo	Produtos, categorias, imagens, disponibilidade e carrinho.
KDS	Concluído como protótipo	Colunas, tickets, tempos e ações simuladas.
JavaScript/DOM	Concluído para a etapa	data.js, components.js, catalog.js, cart.js e demais módulos: arrays, objetos, for...of, for...in, forEach, DOM e cálculos.
Validações	Concluído em nível inicial	validation.js e login.js demonstram validação no navegador; validação de autoridade será repetida no Flask.
Flask/MVC/rotas/JSON	Concluído como estrutura simulada	Aplicação Flask inicial, controllers e endpoints simulados de autenticação, Reservas, Cardápio, Pedidos e KDS.
Segurança e RBAC	Definido e demonstrado no protótipo	Perfis e separação visual existem; autenticação e RBAC de autoridade serão aplicados no servidor.

22 ESTADO ATUAL DA IMPLEMENTAÇÃO E INTEGRAÇÃO

Esta seção consolida a implementação atual sem alterar a arquitetura definida anteriormente. No terceiro bimestre, o front-end e o Flask utilizam dados simulados para demonstrar os fluxos; no quarto bimestre, esses mesmos contratos serão ligados à persistência e à comunicação real por JSON.

22.1 Front-end e JavaScript

O front-end possui áreas separadas para Cliente, Cozinheiro e Administrador. O JavaScript está dividido entre data.js, site.js, components.js, catalog.js, cart.js, validation.js, login.js, views.js e a11y.js, cobrindo arrays, objetos, for...of, for...in, forEach, DOM, validação e cálculos em memória.

22.2 Back-end simulado

A estrutura Flask já demonstra cadastro/login, Bancadas/Reservas, Cardápio/Pedidos e KDS por meio de controllers e models simulados. A finalidade é validar endpoints e responsabilidades antes de conectar o banco e implementar autenticação e RBAC reais.

22.3 Integração e próxima etapa

O fluxo permanece Reserva → Sessão → Cardápio/Carrinho → Pedido → KDS. Na integração definitiva, o JavaScript enviará identificadores e quantidades ao Flask; o servidor validará a Sessão, recalculará valores, persistirá os dados e devolverá JSON. O KDS passará a consumir esses Pedidos e atualizar seus estados pelo back-end.

REFERÊNCIAS

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO. Projeto Integrador Sabor & Clic: sistema de gestão para dark kitchens e espaços gastronômicos. Versão de 31 de agosto de 2026. São Paulo: IFSP, 2026.

SABOR E CLIC. Repositório Sabor-Clic. GitHub, 2026. Disponível em: <https://github.com/FaellEmiliano/Sabor-Clic>.

SABOR E CLIC. Dicionário de Dados Sabor e Clic. São Paulo, 2026.